ASSIGNMENT-02

D D M M Y Y Y Y

1] Define AVL Trees. explain its four rotation types.

→ It is a self balancing binary search tree(BST) in which the difference b/w the height of the left and right subtree of any node is atmost 1. the balance factorial of a node is

Balance factor = Height of left subtree - Height of right tree (Balance factor)

for every node in an AVL Tree.

Balance factor = -1, 0 or 1

If the balance factor becomes less than -1 or greater than +1 after insertion or deletion the tree is rebalanced using rotations

Four rotation types

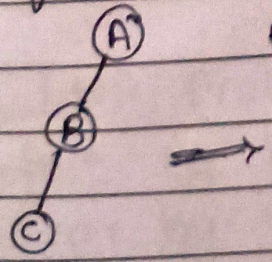
1. LL
2. RR
3. LR
4. RL

Advantages

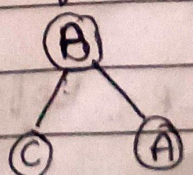
searching O(log n)
insertion O(logn)
deletion O(logn)

1. LL (Left-Left Rotation)

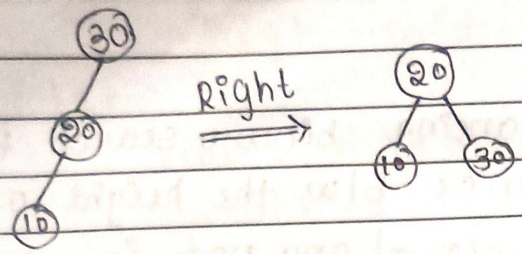
occurs when a node is inserted into the left sub tree of the left child before rotations



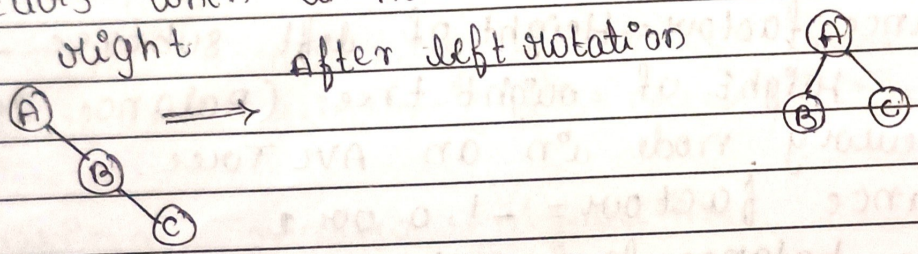

After right rotation



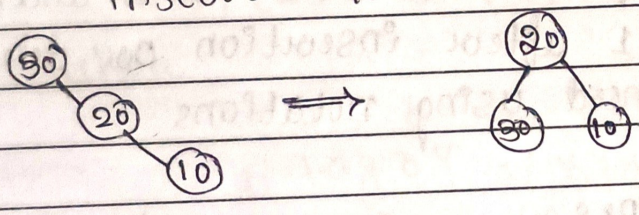
insert 30 20 10



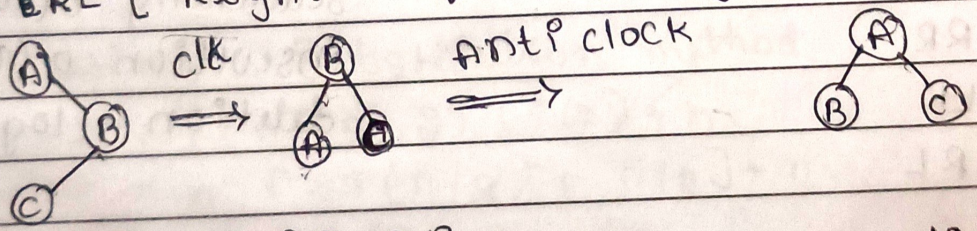

2. RR ( Right - Right Rotation).
occurs when a node is inserted the right subtree of right



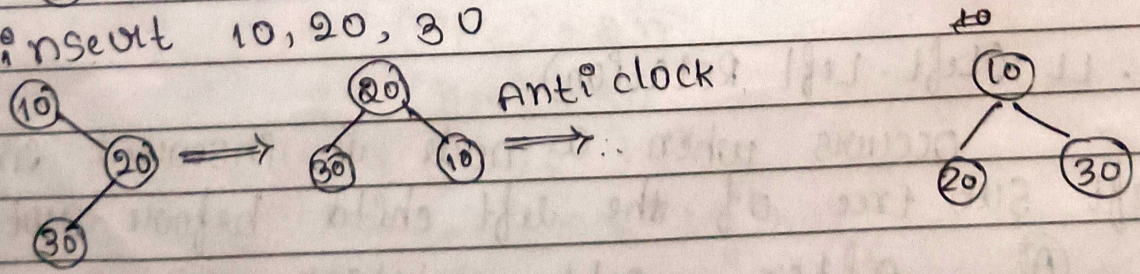

insert 10 20 30



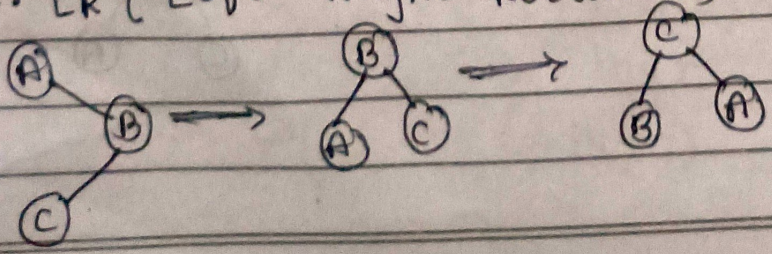

3. LRL [ Right - left rotation )



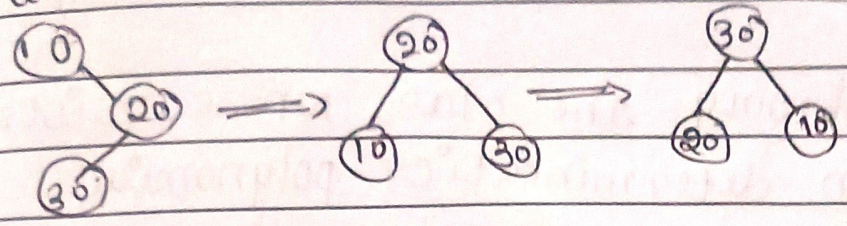

insert 10, 20, 30




4. LR ( Left - Right Rotation )



insert 10 20 30




Explain the following with examples

i) P class problem

→ The set of decision problem that can be solved by deterministic algorithm in polynomial class. The P problem include only decision problem which are problem with yes/NO answers in other words the problem that can be solved in polynomial by deterministic problem are called P problem, or polynomial class (or) p class problems.

The p class problem include only decision problem which are problem with yes/NO answers in other words the problem that can be solved in polyno -mial by determinstic problem are called p problem and

Eg: So searching and sorting problem, finding of gcd of two numbers, check whether the graph is connected acyclic or not finding a minimu -m spanning tree. travelling the shoortest path etc--- are example of p problem.

ii) NP Problem:

defn: the problem that can be solved in polynomial time by non deterministic algorithm are called non deterministic P-type problems (or) NP problems

D D M M Y Y Y Y

N-P means Non-deterministic algorithm polynomial

In this theory the class NP consists of those means non deterministic polynomial.

> In this theory the class NP consists of all those decision problem whose +ve solution can be verified in polynomial time given the right information.
> The solution should be obtained in polynomial time in a non-deterministic machine of the NP class.

iii) NP complete problem:
The decision problem that belogs to NP-class is called NP complete problem.
Every problem in NP is polynomially reduc-iable to decision problem can solved that a decision problem is NP complete into steps.
1] we should showed that the problem in question is in empty i,e randomly generated starting can be checked in polynomial time to determine whether or not it represent the solution to the problem
2] should show that every problem (inputs) in NP is reducible to the problem in questions in polynomial time.

Eg: The following are NP complete problem.
1. Hamiltonium circuits.
2. travelling sales problem

3. Knapsack
4. assignment problem

It is not known whether P= NP or is a just proper subset of empty this question is more important to unresolved issues ~~the~~ theoretical computer science.

A discovery of polynomial time algorithm for any one of Note empty complete problem then P=NP.

ii) NP- Hard Problem:

The optimization version of difficult combinational problems fall on the class of NP Hard problem.
→ The problem that are atleast as hard as NP complete problem are called NP Hard problem
eg: It is difficult to handle problems of combinational optimization such as travelling sale's man problem and knapsack problem. this are known polynomial times algorithm.
for this problem and thes are serious theoretical reason to believe that such algorithm don't exists

8] Define transitive closure of a graph Apply warshalls algorithm to compute transitive closure of a directed graph

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | 1 | 0 | 0 |
| b | 0 | 0 | 0 | 1 |
| c | 0 | 0 | 0 | 0 |
| d | 1 | 0 | 1 | 0 |

→ The transitive closure of a directed graph is the graph obtained by adding edges b/w all pairs of vertices for which there exists a path from u to v in G.

Soln: The adjacency matrix for the given graph.

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | 1 | 0 | 0 |
| b | 0 | 0 | 0 | 1 |
| c | 0 | 0 | 0 | 0 |
| d | 1 | 0 | 1 | 0 |

step 1: The path through vertex a, since we finding the path through a 4th row and 4th column need to be considered

$(d,a) = 1 + (a,b) = 1$

so, $(d,b) = 1$

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | 1 | 0 | 0 |
| b | 0 | 0 | 0 | 1 |
| c | 0 | 0 | 0 | 0 |
| d | 1 | 1 | 1 | 0 |

step 2: the path through vertex b, since we are finding the path through b, bth row & bth column need to be consider

Since $(a,b) = 1$ and $(b,d) = 1$, $(a,d) = 1$

Since $(d,b) = 1$ and $(b,1) = 1$ $(d,d) = 1$

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | 1 | 0 | 1 |
| b | 0 | 0 | 0 | 1 |
| c | 0 | 0 | 0 | 0 |
| d | 1 | 1 | 1 | 1 |

Step3: The path through vertex c, since we are finding the path through c, cth row and cth column need to be considered

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | 1 | 0 | 1 |
| b | 0 | 0 | 0 | 1 |
| c | 0 | 0 | 0 | 0 |
| d | 1 | 1 | 1 | 1 |

(d, c) = 1 from vertex c there is not path to any other node

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | 1 | 0 | 1 |
| b | 0 | 0 | 0 | 1 |
| c | 0 | 0 | 0 | 0 |
| d | 1 | 1 | 1 | 1 |

Step4: the path through vertex d since we are finding the path through

| | a | b | c | d |
|---|---|---|---|---|
| a | 0 | 1 | 0 | 1 |
| b | 0 | 0 | 0 | 1 |
| c | 0 | 0 | 0 | 0 |
| d | 1 | 1 | 1 | 1 |

Since (a,d) = 1 (d,a) = 1 so (a,a) = 1

(a,d) = 1 (d,b) = 1 (a,b) = 1

(a,d) = 1 and (d,c) = 1 S₀ (a,c) = 1

(d,d) = 1 and (d,a) = 1 S₀ (a,d) = 1

(b,d) = 1 and (b,a) = 1 S₀ (b,a) = 1

(b,d) = 1 and (d,b) = 1 S₀ (b,b) = 1

(b,d) = 1 and (d,c) = 1 S₀ (b,b) = 1

(b,d) = 1 and (d,d) = 1 S₀ (d,d) = 1

| | a | b | c | d |
|---|---|---|---|---|
| a | 1 | 1 | 1 | 1 |
| b | 1 | 1 | 1 | 1 |
| c | 0 | 0 | 0 | 0 |
| d | 1 | 1 | 1 | 1 |

14] Explain Branch and Bound knapsack Approximation Algorithm.

→ The idea of backtracking is use to solve optimization Problems that are usually subject to some constraints by specifying either lower bound / upper bound.

By specifying the lower bound we can solve the Problems that seek to maximize an objective function.

feasible solution: A feasible solution is a solution is a problem search space that specify the given constraints.

Eg: travelling saliman problem.

D D M M Y Y Y Y

Optimal Solution: An optimal solution is a feasible soln which maximize the profit and maximize the loss with best value of the objective solution.

Algorithm Knapsack Approximation.

Start
Input no of item n
Input weight w[i] and profit P[i]
Input knapsack capacity w
Computer ratio P[i] w[i]
Sort item in descending order of ratio
Initialize total_profit = 0
for each item
if w[i] <= w
Select the item
total_profit = 0
for each item:
if w[i] <= w:
Select the item
total_profit t = P[i]
w = w - w[i]
Display Selected Items and total Profit
Stop.

D D M M Y Y Y Y

Explain space and trade offs. Give the algorithm for sorting by comparision counting. sort the given arrays using sorting comparision, 24.45, 10, 20, 15, 10

⇒ An algorithm must with be time efficient and space efficient - to achieve both may not be possible for some of the algorithm in some suitation. Space may be important factorial. and in someother suitable time may be important factorial. thus important factor space and time tradioff is a suitation in which thus either time efficiency can be achieve at the cost of extra memory usage or space efficiency can be achieve at the cost of the execution speed. their ~~Should~~ should be compromize blw time and space based on the suitation.

Algorithm: comparison - counting (a, n, b)
// purpose: To sort n elements stored in an arraya

//input: a the item to be sort if n - the no of items to be sort.

//output:

b: contain the sorted list.

for i ← 0 to n-1 do
c[i] ← 0 // initialize count of each element
for i ← 0 to n-2 do // compute no of elements.
for j ← i+1 to n-1 do // use than each element.
if (a[i] < a[j])

```
        c[i] ← [j] + 1;
    else
        c[i] ← c[i] + 1
    end four
    end four
end four.
```

Let us taken an example consider the element shown as below

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| a. | 25 | 45 | 10 | 20 | 50 | 15 |

Step 1: compute the no of element less than each element as shown below.

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| a | 24 | 45 | 10 | 20 | 50 | 15 |

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| C | 3 | 4 | 0 | 2 | 5 | 1 |

→ NO of elements less than 15

→ NO of elements less than 50

→ NO of elements less than 20

→ NO of elements less than 10

→ NO of elements less than 45

→ NO of elements less than 25

D D M M Y Y Y Y

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| a | 25 | 45 | 10 | 20 | 50 | 15 |

| index | 0 | 1 | 3 | 4 | 5 | 5 |
|---|---|---|---|---|---|---|

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| when i=0 | 0 | 1 | 2 | 3 | 4 | 5 |
| i, e pass=0 | 3 | 1 | 0 | 0 | 1 | 0 |
| when, i = 1 | | 4 | 0 | 0 | 2 | 0 |
| ~~i, e pass=1~~ | | | | | | |
| when i = 2 | | | 0 | 1 | 3 | 1 |
| i, e pass=2 | | | | 2 | 4 | 0 |
| when i=3 | | | | | | |
| i, e pass=3 | | | | | 5 | 1 |

when i=4
i, e pass=4

| c | 3 | 4 | 0 | 2 | 5 | 1 |
|---|---|---|---|---|---|---|

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| | 10 | 15 | 20 | 25 | 45 | 50 |

Sorted list

15] Explain Decision tree with an example.

→ A Decision tree also called comparision tree it is a binary tree that represent only the comparison of given element in the array. while searching in the algorithm or sorting in the algorithm the control transfer data moment and all other ascept of the algorithms are ignore.

⇒ In this decision true the internal node represent the comparision may between the items where as the leaf node represent the result of the algorithm.

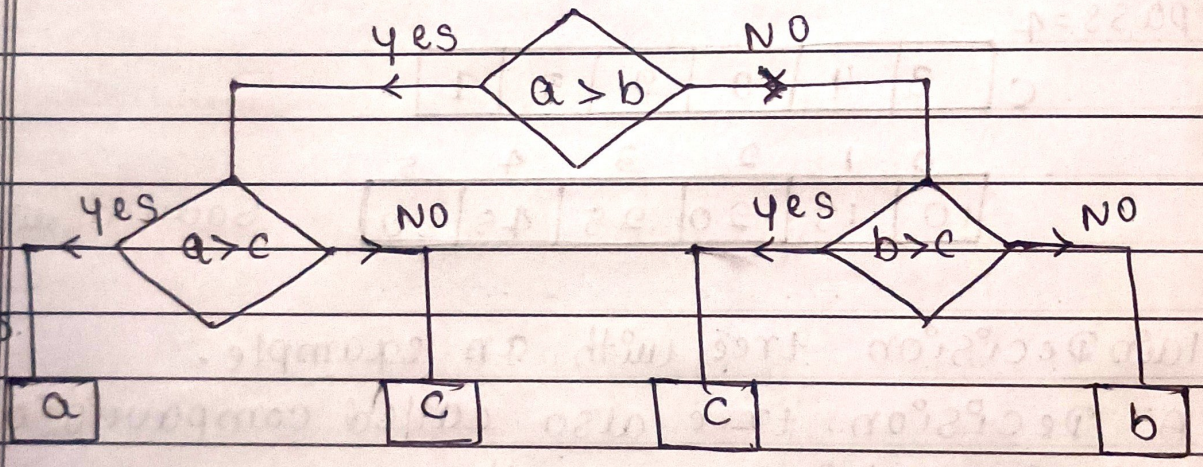
D D M M Y Y Y Y

obtaining the decision tree to find the min of 3 numbers.

→ The decision tree are comparision tree so only the comparision of two elements in each of the node

→ The node of the leaf level are the main number are obtained

→ The decision tree to find min of 3 number is shown below.




If a binary tree of height n has max number of leaves. then it has only the leaves on the last level the number of nodes each level are the number of nodes in level $0 = 1 = 2^0$

$1 = 2 = 2^1$

$2 = 4 = 2^2$

let the no of nodes in last level

$2^n \geq 2$

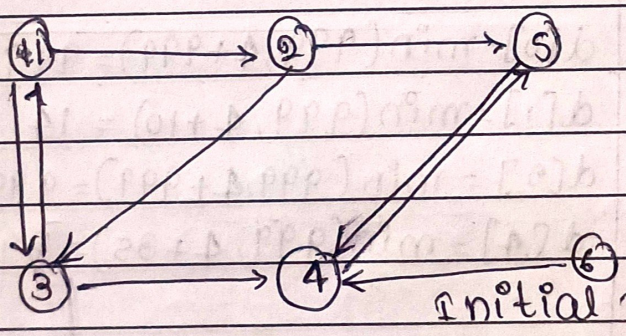
taking log on both side

$$\log 2^h \geq \log 1$$
$$h \log 2^2 \geq \log 2\ 1$$
$$h_1 \geq \log 2\ 1$$

$$\therefore \boxed{h \geq \log_2 1}$$

the observe that the lower bound depends on the height of binary decision tree such a bound is called information theoruty bound tree.

7] Apply Dijkstra's algorithm to find single source shortest path for the given graph by considering 8 as the source vertex




Initial matrix

→ cost

| | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | 0 | 15 | 10 | 999 | ~~45~~ | 999 |
| 2 | 999 | 0 | 15 | 999 | 20 | 999 |
| 3 | 20 | 999 | 0 | 20 | 999 | 0 |
| 4 | 999 | 10 | 999 | 0 | 35 | 999 |
| 5 | 999 | 999 | 999 | 30 | 0 | 999 |
| 6 | 999 | 999 | 999 | 4 | 999 | 0 |

D D M M Y Y Y Y

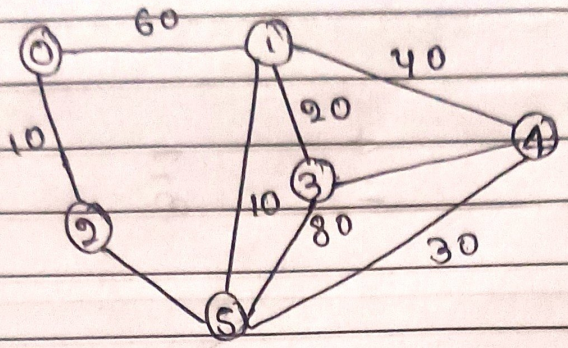
| | A | B | C | D |
|---|---|---|---|---|
| d[1] | 999 | 999 | 999 | 49 |
| d[2] | 999 | 14 | 14 | 14 |
| d[3] | 999 | 999 | 29 | 29 |
| d[4] | 4 | 4 | 4 | 4 |
| d[5] | 999 | 39 | 34 | 34 |
| d[6] | 0 | 0 | 0 | 0 |

Tracing table :-

| S | S=V-S | d[v] = min (d(v), d(v)+w(v,v)] | d(v)→min |
|---|---|---|---|
| | | | 4,4 |
| 6 | 1,2,3,45 | | |
| 6,4 | 1,2,3,5 | d[0]=min(999,4+999)=999 | 2,14 |
| | | d[1]=min(999,4+10) = 14 | |
| | | d[2] = min(999,4+999)=999 | |
| | | d[4] = min(999,4+35)=39 | |
| 6,4,2 | 1,3,5 | d[0] = min(999,14+999)=999 | 3,29 |
| | | d[2] = min(999,14+15)=29 | |
| | | d[4] = min(39,14+20)=34 | |
| 6,4,2,3 | 1,5 | d[0]=min(999,29+20)=49 | 5, |
| | | d[4]=min(34,29+999)=34 | 5,3,4 |
| 6,4,23 | 1 | d[0]=min(49,34+999)=49 | 1,49 |
| 5 | | | |

D D M M Y Y Y Y

12] Prims problem




| | | | | | |
|---|---|---|---|---|---|
| 0 | 60 | 10 | ∞ | ∞ | ∞ |
| 60 | 0 | ∞ | 20 | 40 | 70 |
| 10 | ∞ | 0 | ∞ | ∞ | 50 |
| ∞ | 20 | ∞ | 0 | ∞ | 80 |
| ∞ | 40 | ∞ | ∞ | ∞ | 30 |
| ∞ | 70 | 50 | 80 | 30 | 0 |

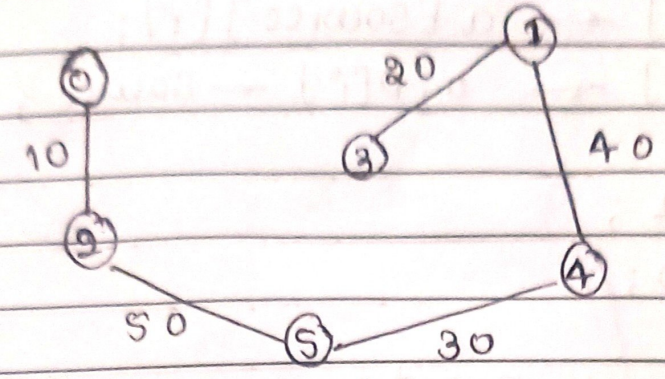
→

| | d |
|---|---|
| 0 | 0 |
| 1 | 60 |
| 2 | 10 |
| 3 | ∞ |
| 4 | ∞ |
| 5 | ∞ |

| | P |
|---|---|
| 0 | 0 |
| 1 | 0 |
| 2 | 0 |
| 3 | 0 |
| 4 | 0 |
| 5 | 0 |

| S | V=V-S | d[v]=min[d[v] cost[(u)(v)] | P(k)=4 | 4 9 d[4] | output 4, P[4] |
|---|---|---|---|---|---|
| 0 | 1,2,3,4,5 | | | | |
| 0,2 | 1,3,4,5 | d[1]=min(0,∞) = 60<br>d[3]=min(∞,∞) = ∞<br>d[4]=min(∞,∞)=∞<br>d[5]=min(∞,50)=50 | | 2,10 | 2,0 |
| | | | P(5)=2 | 5,50 | 5,2 |
| 0,2,5 | 1,3,4 | d[1]=min(60,70)=60<br>d[3]=min(∞,80)=80<br>d[4]=min(∞,30)=30 | P[3]=5<br>P[4]=5 | 4,30 | 4,5 |
| 0,2,5,4 | 1,3 | d[1]=min(60,40)=40<br>d[3]=min(80,∞)=80 | P[1]=4 | 1,40 | 1,4 |
| 0,1,2,5,4 | 3 | d[3]=min(80,20)=0 | P[3]=1 | 3,20 | 3,1 |
| 0,1,2,3,4,5 | - | - | - | - | - |

(2,0) → (5,2) → (4,5) → (1,4) → (3,1)




The cost of minimum spanning tree

= 10+50+30+40+20 = 150

19] write prims algorithmand give its time complexity.

ALGORITHM PRIM(n,w)
Input: n - Number of vertices
w - cost adjecency matrix with value >=0
output: spanning tree if exists

```
min ← 9999;
Source ← 0;
for i ← 0 to n-1 do
    for j ← 0 to n-1 do
        if [a[i][j] != 0 && a[i][j] <= min)
            min ← a[i][j];
            source ← i;
    end if
  end for
end for
```

```
for i ← 0 to n-1 do
        d[i] ← a[source][i];
        s[i] ← 0, p[i] ← source;
end for
s[source] ← 1;
    sum ← 0;
     k ← 0;
    for i ← 0 to n-1 do
      Find u and d[u] such that d[u] is
       minimum and u in V-S
       Add u to S
       Select an edge with least cost
       Add cost to selected edge to get total
       cost of minimum spanning tree

 for every v belongs to V-S do
  if (a[u][v]<d[v])
        d[v] ← a[u][v];
        p[v] ← u;
 end if
  end for
 if (sum ≥ 9999)
        write "spanning tree does not exist";
 end else
        write "spanning tree exists and minimum
              spanning tree is",
 endif
```

Time complexity

12] using Branch and Bound technique solve the below instance of knapsack problem.

| Item | weight | value |
|---|---|---|
| 1 | 4 | 40 |
| 2 | 7 | 42 |
| 3 | 5 | 25 |
| 4 | 3 | 12 |

capacity 5

The given data is shown below

| weight | value | v/w |
|---|---|---|
| $w_1 = 4$ | $v_1 = 40$ | $v_1 / w_1 = 10$ |
| $w_2 = 7$ | $v_2 = 42$ | $v_2 / w_2 = 6$ |
| $w_3 = 5$ | $v_3 = 25$ | $v_3 / w_3 = 5$ |
| $w_4 = 3$ | $v_4 = 12$ | $v_4 / w_4 = 4$ |

D D M M Y Y Y Y

The capacity of knapsack given m=5 the upper bound is calculated using the following relation.

$$V = v + (m-w)\, v_{i+1} / w_{i+1}$$

∴ v is the Porofit

∴ w is the weight

item i=0; initially state free

$v_1 / w_1 = 10$

$w = 0 \quad v = 0$

$$UB = v + (m-v)\, v_1 / w_1$$
$$= 0 + (5-0) \times 10$$
$$= 50$$

| $w = 0$ | $v = 0$ |
|---|---|
| $UB = 50$ | |

with item 1 $v_2 / w_2 = 6$

$w = 0 + 4 = 4$

$v = 0 + 40 = 40$

$m - w = 5 - 4 = 1$

$$UB = v + (m-v)\, v_2 / v_2$$
$$= 40 + 1 \times 6$$
$$UB = 46$$

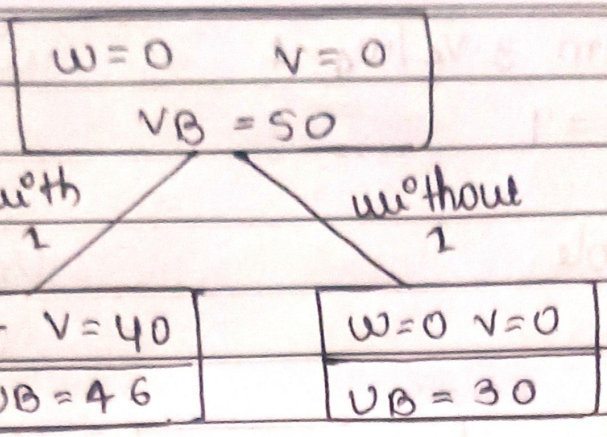
without item 1

$w = 0 \quad v = 0$

$v_2 / w_2 = 6$

$UB = 0 + 5 \times 6$

$UB = 30$




with item 2 $v_3/w_3 = 5$

$w = 4 + 7 = 11$

since

$11 > 5$

Not feasible

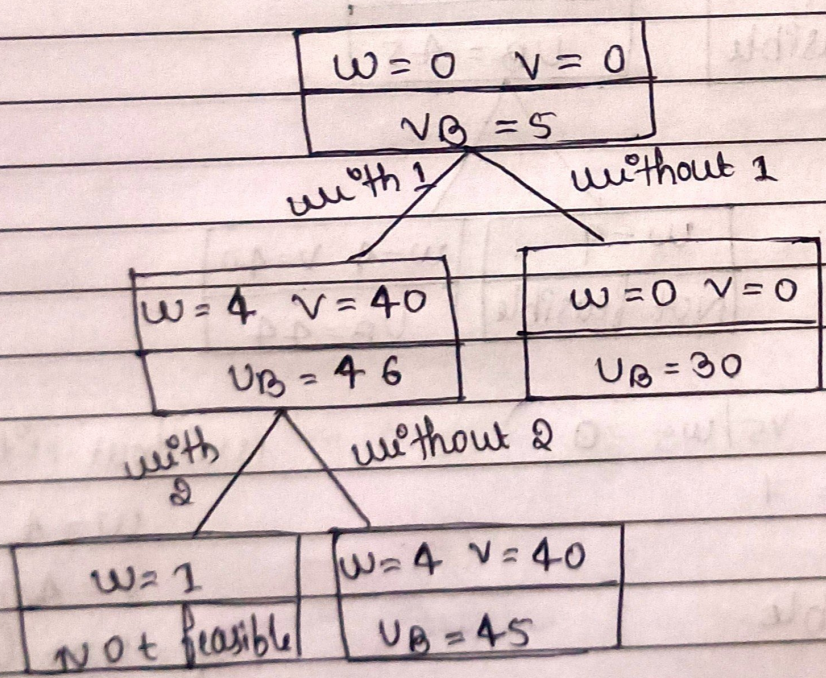
without item 2

$w = 4$ $v_3/w_3 = 5$

$v = 40$

$U_B = 40 + (5-4) \times 5$

$U_B = 45$



D D M M Y Y Y Y

with item 3 $v_4/w_4 = 4$

$w = 4 + 5 = 9$

$9 > 5$

Not feasible

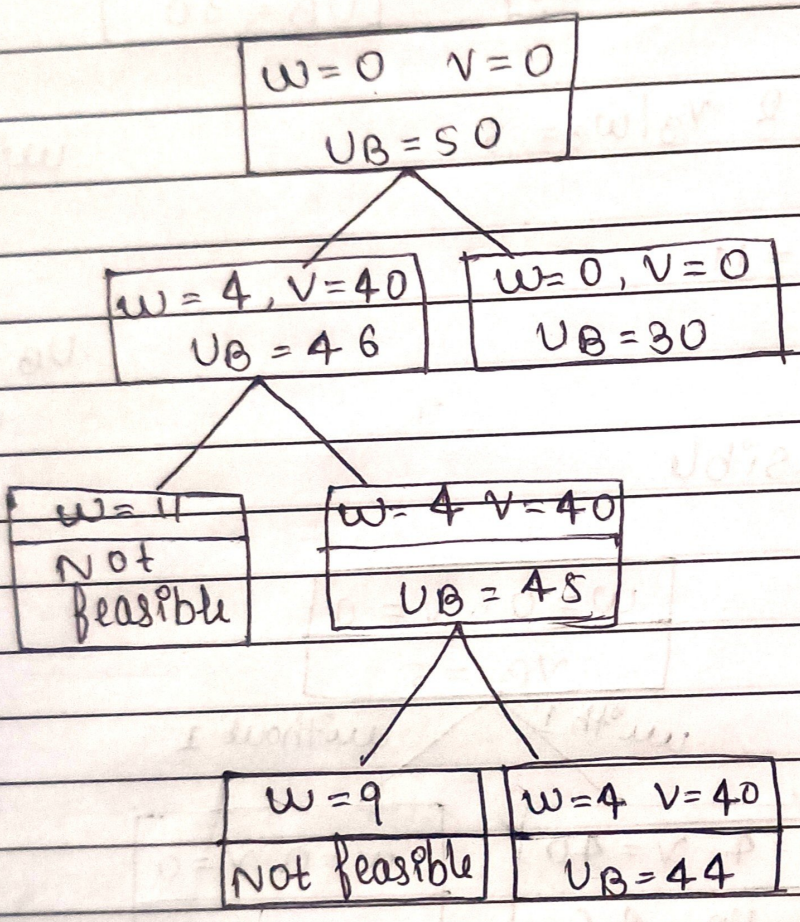
without item 3 $v_4/w_4 = 4$

$w = 4$

$v = 40$

$U_B = 40 + (5-4) \times 4$

$U_B = 44$




with item 4 $v_5/w_5 = 0$

$w = 4 + 3 = 7$

$7 > 5$

not feasible

without item 4 $v_5/w_5 =$

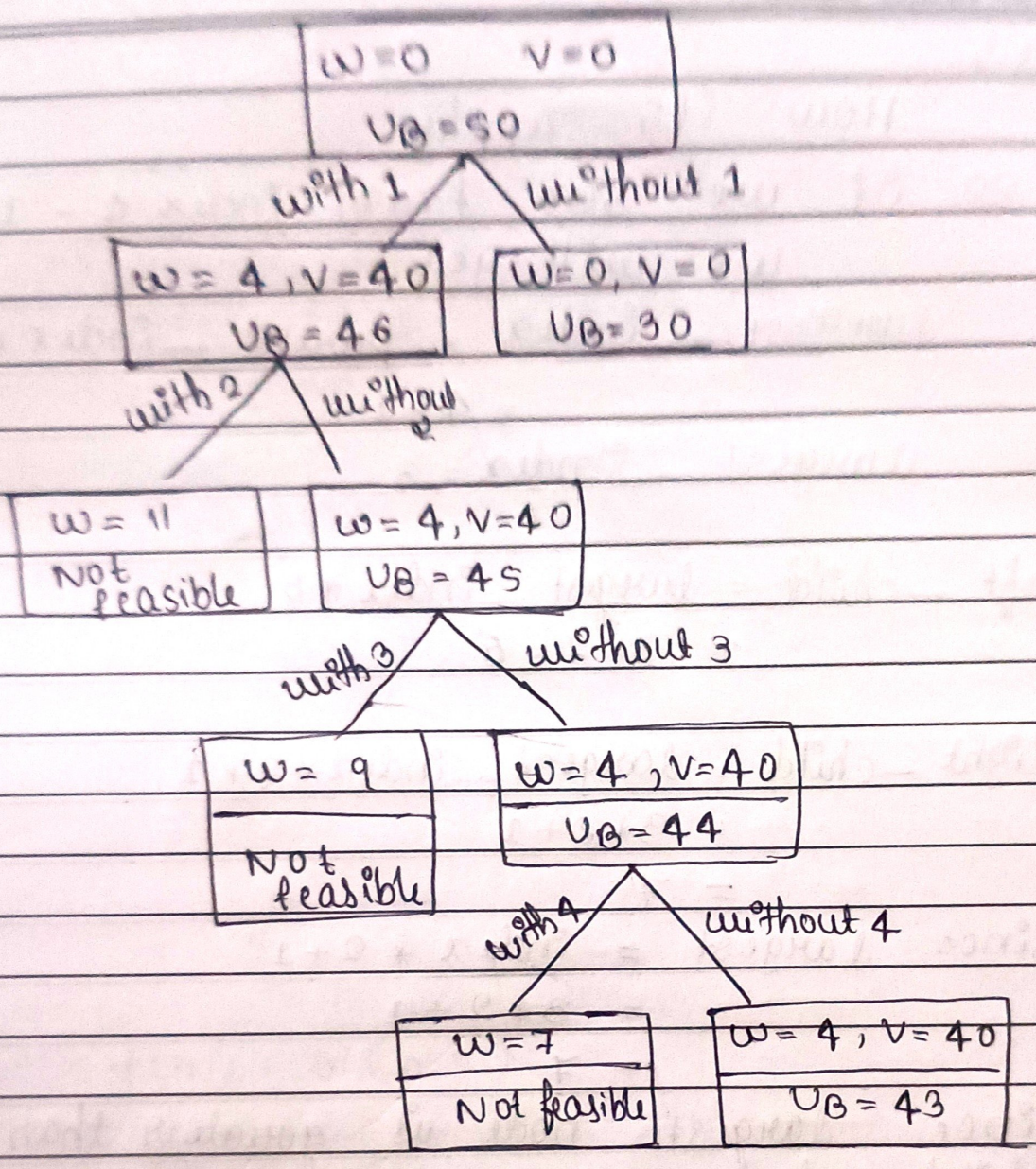
$w = 4$

$v = 40$

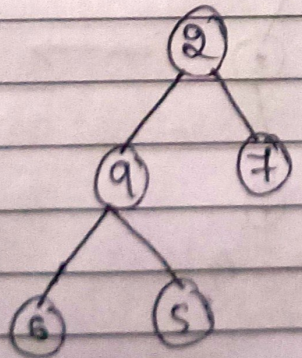
$U_B = 40 + (5-4) \times 0$

$= 40 + 0$

$U_B = 40$




2] Construct bottom up heap for the list 2, 9, 7, 6, 5, 8 obtain its time complexity Heap construction



Sol:-

How its apply

So if we sub from index 4 - 1

we will get

largest_index = parent_index * 2

= 4 - 1

largest_index = 3

left_child = largest_index * 2

= 3 × 2 = 6

Right_child = largest_index * 2 + 1

= 3 * 2 + 1

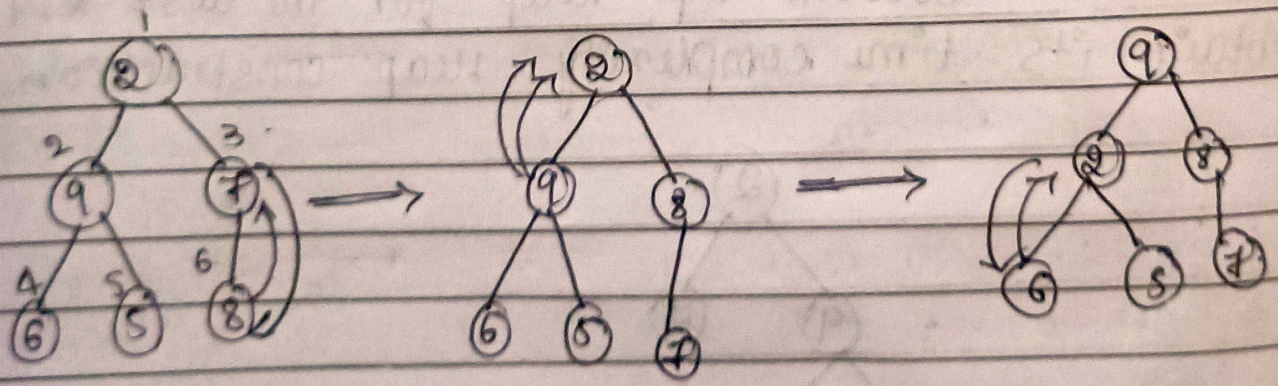
= 7

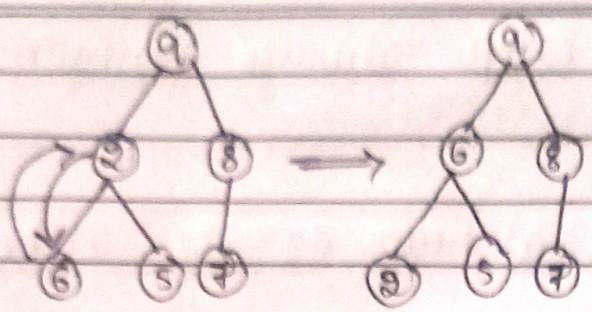
Since largest = index * 2 + 1

= 3 * 2 + 1

= 7

since largest node is greater than both child node so no swapping is required.





9 is maximum number

= Heap tree

Time complexity

$$T(n) = \sum_{n=0}^{\log_2 n} \boxed{\frac{n}{2^{n+1}}} \quad O(h)$$

$$= 0\left(n \sum_{n=0}^{\infty} \frac{n}{2^n}\right)$$

$$= \sum_{n=0}^{\infty} \frac{n}{2^n}$$

$$T(n) = 0(n.2)$$

$$\boxed{T(n) = O(n)}$$

3] Define Heap. Explain the properties of heap along with its representation.

→ Heap is a complete binary tree or a almost complete binary tree in which every parent node be either greater or lesser than its child node

Properties of heap:

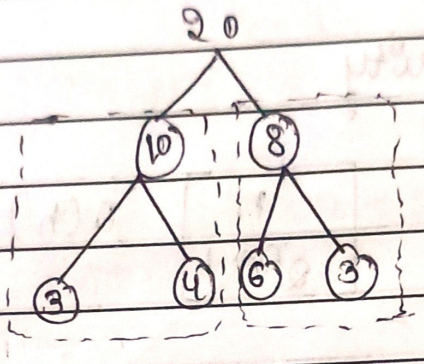
1] there should be either " complete binary tree " or " almost complete tree ".

D D M M Y Y Y Y

2] The root of a heap always contains its largest element.

3] each subtree in a heap is also a heap

eg:




4] Heap can be implemented as array by recording its element in the top down, left to right fashion: represent index

For array A(size)

→ Root node is sorted at A[0]

→ left child is at i+1 position in array A

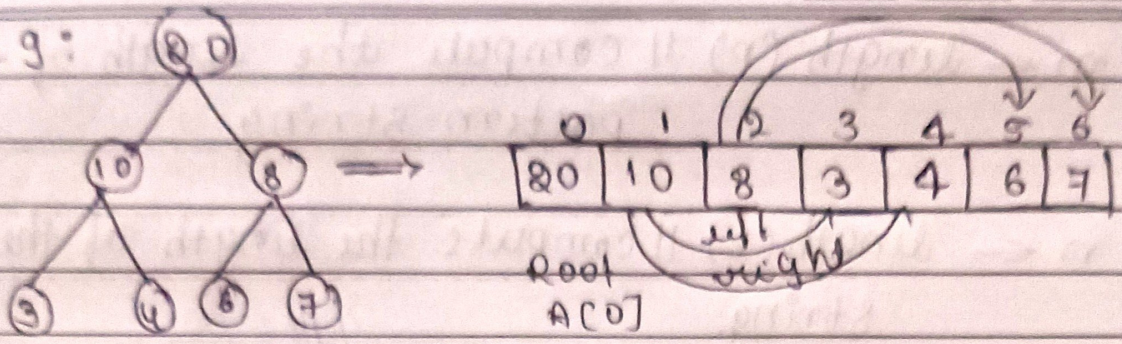
→ Right child is at 2i+2 position in array A

→ Parent node is at (i-1) position in Array A

eg.

D D M M Y Y Y Y

Eg:




Thus array can represent compute heap.

4] Design Horspools algorithm for string matching. Apply Horspools algorithm to find the patter BABBER in the text. JIM_SAW_ME_IN_A BABBERSHOP 1 0

→ Algorithm Horspools - pattern - matching

//Purpose: To check whether the pattern string is present in the text string

//Input: p - pattern string to be searched
t - text string where searching take place

//output: position of pattern string p in the text string
r if successful - 1 otherwise

//compute the necessary shift to be taken
shift-table(P,s) // s is the table which contains the required shift.

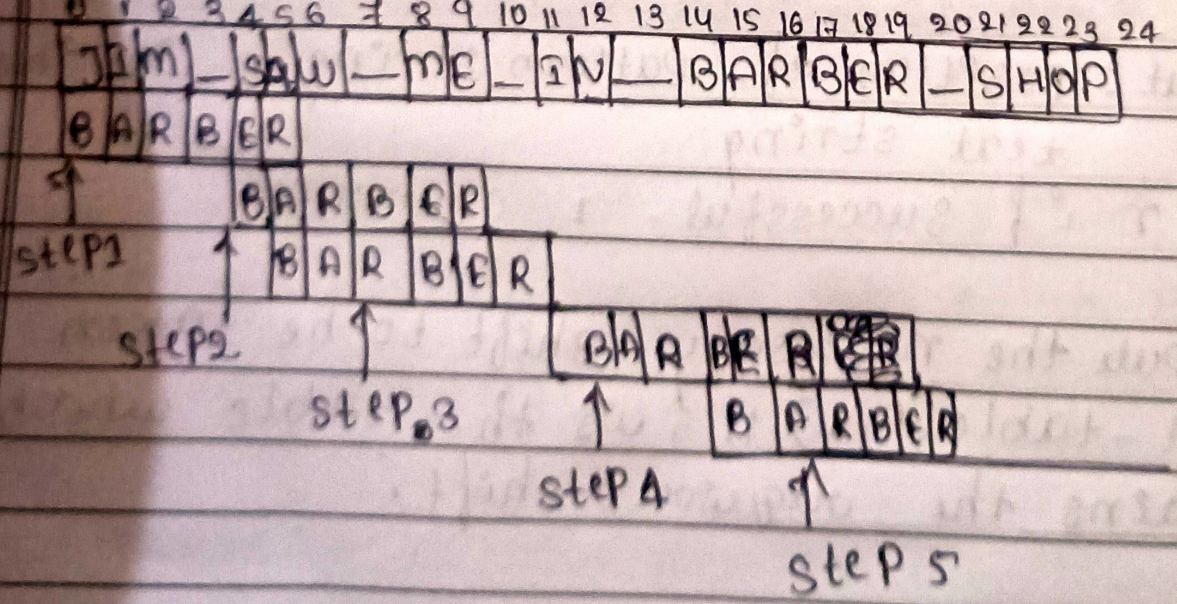
m ← length (P) // compute the length of the pattern string.

n ← length (t) // compute the length of the text string.

i ← m-1 // position of pattern string at the beginning of the text string.

//search for the pattern string in the text string
while (i <= n-1)
    k ← 0
while (k <= m-1 and t[i-k] = P[m-1-k]) do
    k ← k+1 // matching in process.
end while
if (k = m) return i-m+1
    i ← i+S[t(i)] // allign pattern string using text further.

return -1 // pattern not found.



D D M M Y Y Y Y

Step 1: Align BARBER to the beginning of the text-string. the character A and R is not equal. So take is H so

Step 2: compare E and R they are not equals so, take character E and use it as index in shift table S i,e S[E] is 10 so shift the pattern string towards right by 1 position.

Step 3: compare symbol '_' with R, they are not equal. So take character '_' end use it an the index in shift table S i.e S[-] is 6, so shift pattern string towards right by 6 positions.

Step 4: compire R and R they are equal so move left and compare A and E they are not equal. Now take R last character if substring in the text string e and use it an the index in shift table S i,e S[R] in 2 so, shift pattern string towards right by 2 positions.

Step 5: After comparing it is found that all character of pattern string are found and hence there is match, so pattern string found.